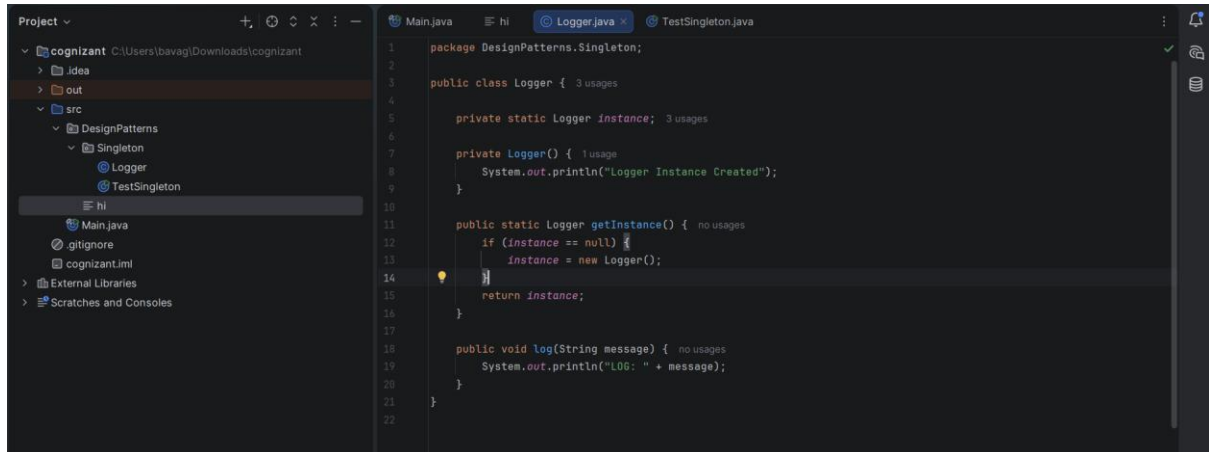


MODULE-1 {DESIGN PATTERNS}

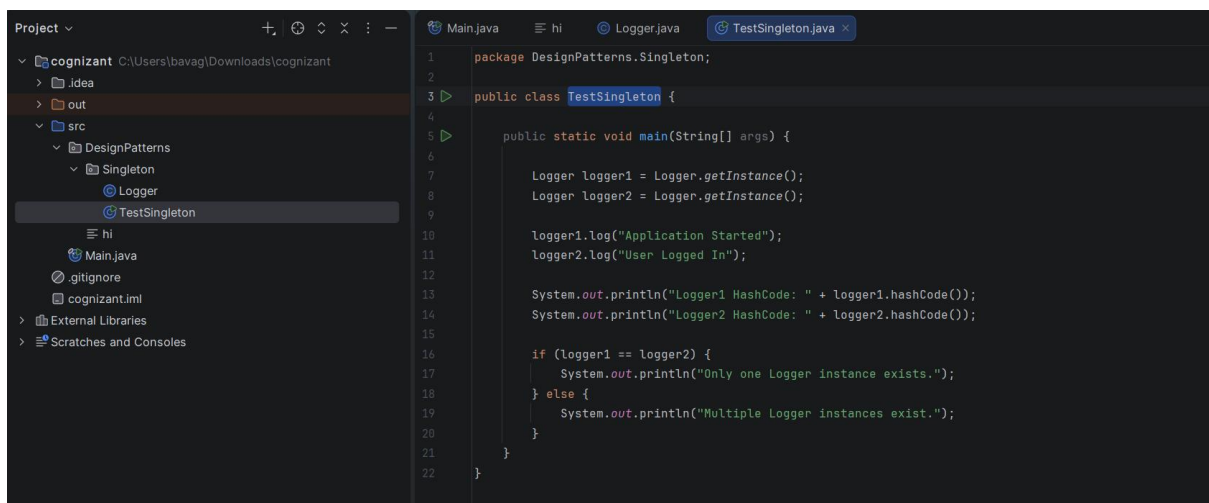
Exercise 1: Implementing the Singleton Pattern

Logger.java



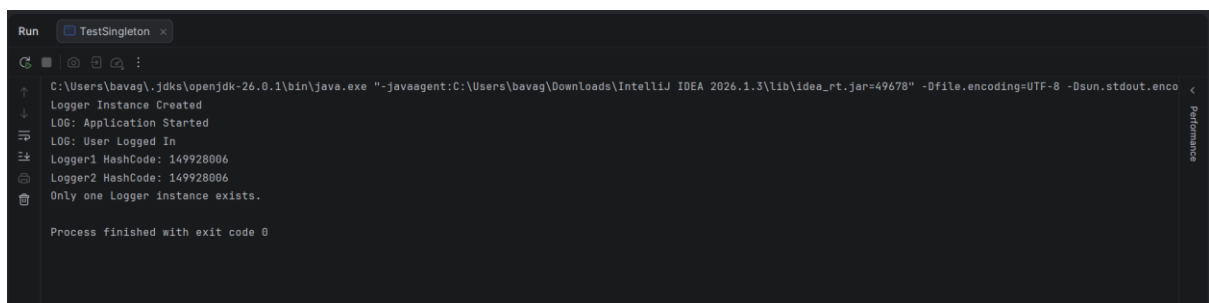
```
1 package DesignPatterns.Singleton;
2
3 public class Logger {
4
5     private static Logger instance;
6
7     private Logger() {
8         System.out.println("Logger Instance Created");
9     }
10
11     public static Logger getInstance() {
12         if (instance == null) {
13             instance = new Logger();
14         }
15         return instance;
16     }
17
18     public void log(String message) {
19         System.out.println("LOG: " + message);
20     }
21 }
22
```

TestSingleton.java



```
1 package DesignPatterns.Singleton;
2
3 public class TestSingleton {
4
5     public static void main(String[] args) {
6
7         Logger logger1 = Logger.getInstance();
8         Logger logger2 = Logger.getInstance();
9
10        logger1.log("Application Started");
11        logger2.log("User Logged In");
12
13        System.out.println("Logger1 HashCode: " + logger1.hashCode());
14        System.out.println("Logger2 HashCode: " + logger2.hashCode());
15
16        if (logger1 == logger2) {
17            System.out.println("Only one Logger instance exists.");
18        } else {
19            System.out.println("Multiple Logger instances exist.");
20        }
21    }
22 }
```

Output:

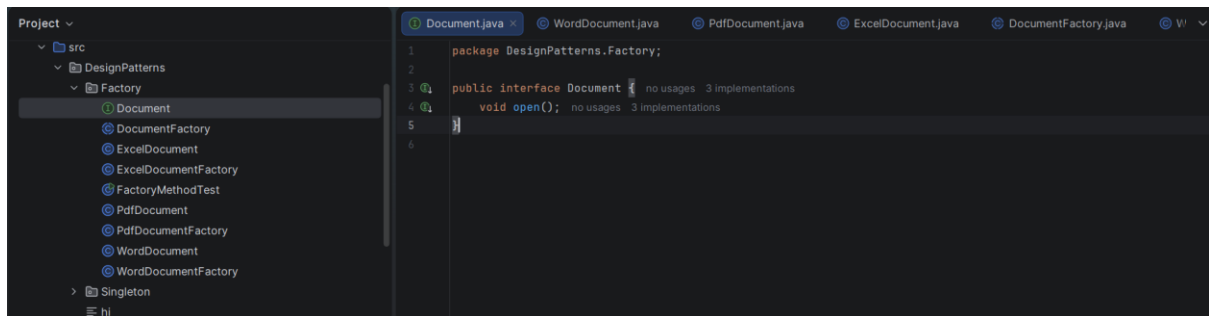


```
Run TestSingleton
C:\Users\bavag\jdk-26.0.1\bin\java.exe "-javaagent:C:\Users\bavag\Downloads\IntelliJ IDEA 2026.1.3\lib\idea_rt.jar=49678" -Dfile.encoding=UTF-8 -Dsun.stdout.enco
Logger Instance Created
LOG: Application Started
LOG: User Logged In
Logger1 HashCode: 149928006
Logger2 HashCode: 149928006
Only one Logger instance exists.

Process finished with exit code 0
```

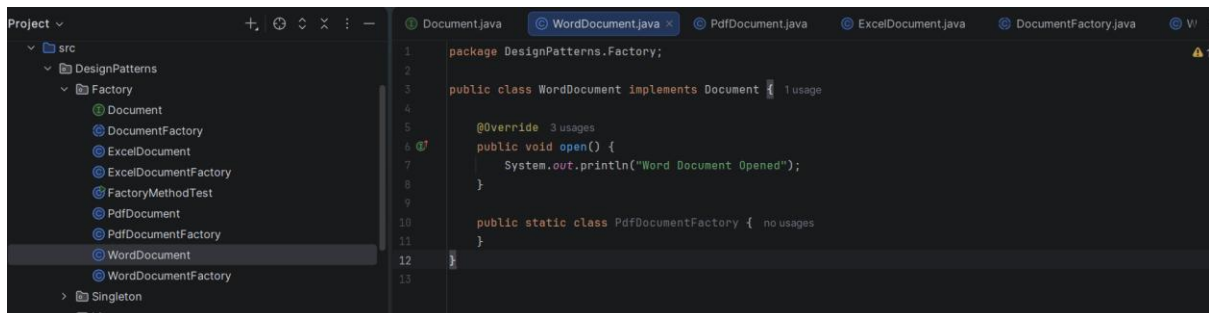
Exercise 2: Implementing the Factory Method Pattern

Document.java



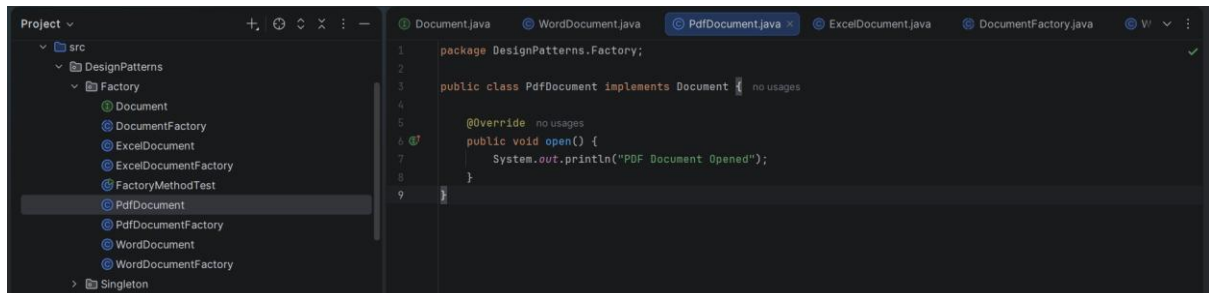
```
1 package DesignPatterns.Factory;
2
3 public interface Document {
4     void open();
5 }
6
```

WordDocument.java



```
1 package DesignPatterns.Factory;
2
3 public class WordDocument implements Document {
4
5     @Override
6     public void open() {
7         System.out.println("Word Document Opened");
8     }
9
10     public static class PdfDocumentFactory {
11     }
12 }
13
```

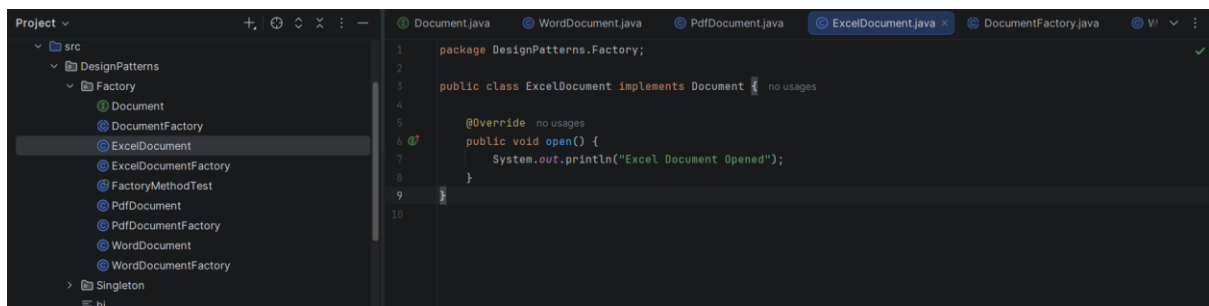
PdfDocument.java



```
1 package DesignPatterns.Factory;
2
3 public class PdfDocument implements Document {
4
5     @Override
6     public void open() {
7         System.out.println("PDF Document Opened");
8     }
9 }

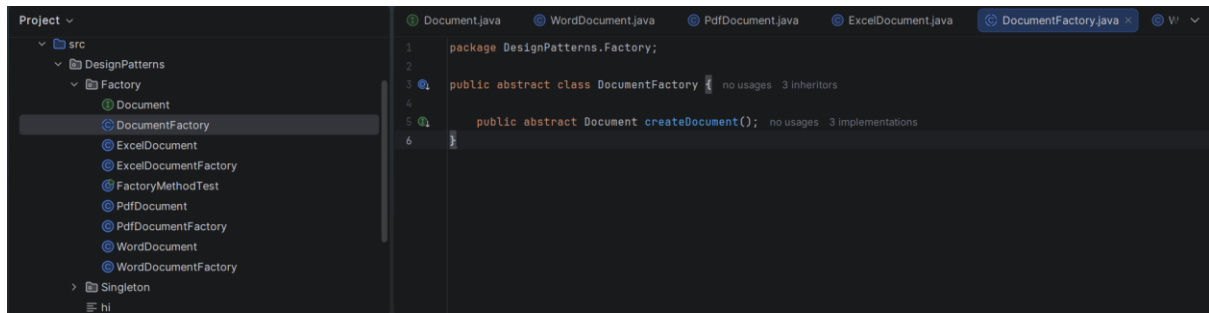
```

ExcelDocument.java

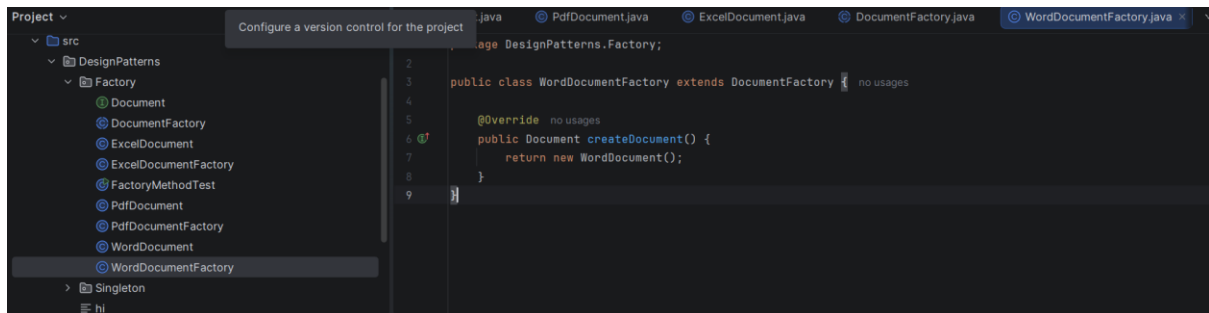


```
1 package DesignPatterns.Factory;
2
3 public class ExcelDocument implements Document {
4
5     @Override
6     public void open() {
7         System.out.println("Excel Document Opened");
8     }
9 }
10
```

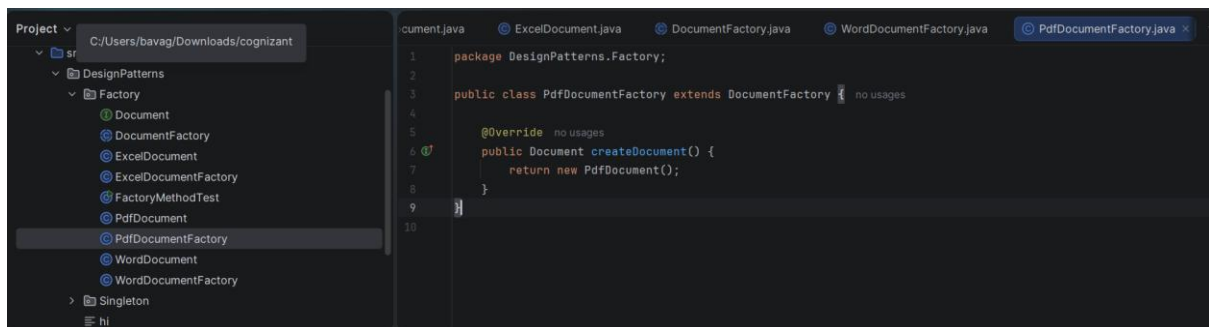
DocumentFactory.java



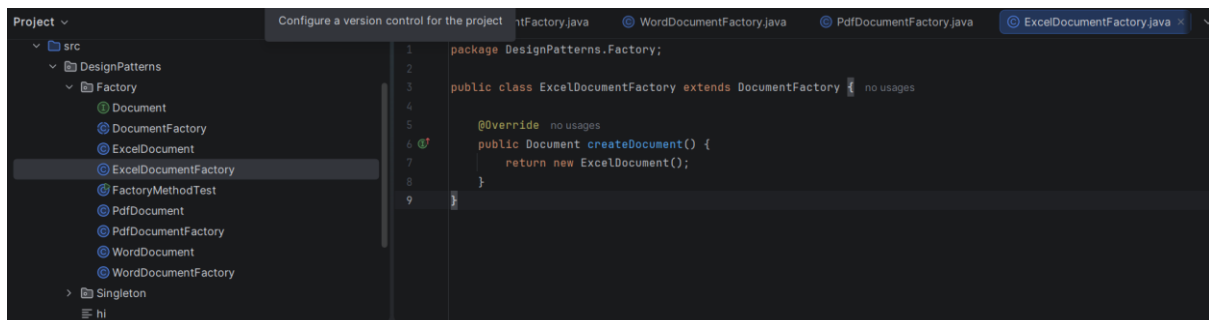
WordDocumentFactory.java



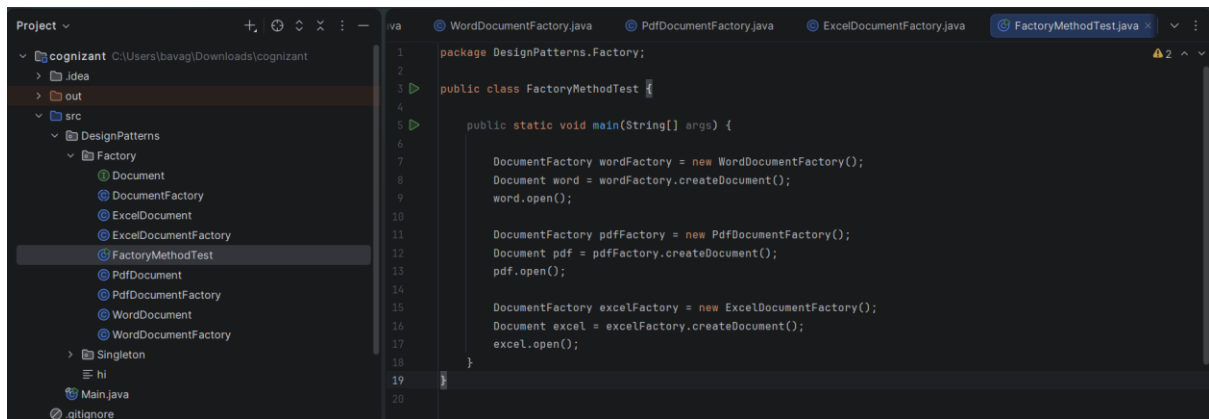
PdfDocumentFactory.java



ExcelDocumentFactory.java



FactoryMethodTest.java

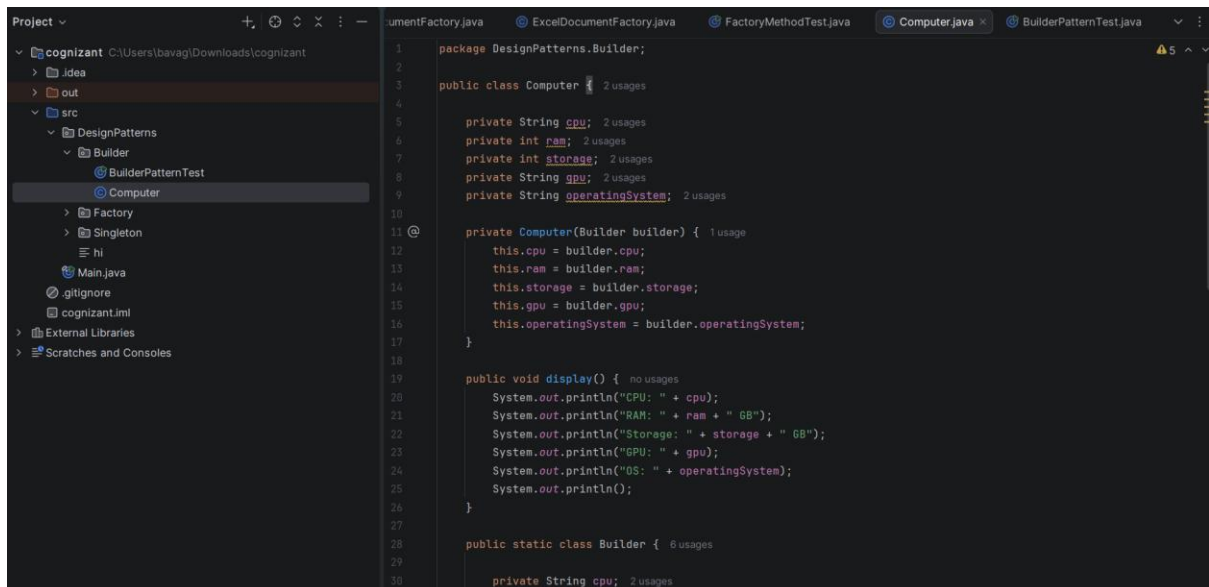


Output:



Exercise 3: Implementing the Builder Pattern

Computer.java



BuilderPatternTest.java

Output

```
Run BuilderPatternTest x
C:\Users\bavag\.jdk\openjdk-26.0.1\bin\java.exe "-javaagent:C:\Users\bavag\Downloads\IntelliJ IDEA 2026.1.3\lib\idea_rt.jar=54670" -Dfile.encoding=UTF-8 -Dsun.stdout.enco
CPU: Intel i9
RAM: 32 GB
Storage: 1000 GB
GPU: RTX 4080
OS: Windows 11
CPU: Intel i5
RAM: 16 GB
Storage: 512 GB
GPU: null
OS: Windows 10
```

Exercise 4: Implementing the Adapter Pattern

PaymentProcessor.java

```
Project
  cognizant C:\Users\bavag\Downloads\cognizant
    > idea
    > out
    > src
      > DesignPatterns
        > Adapter
          > AdapterPatternTest
          > PaymentProcessor
          > PayPalAdapter
          > PayPalGateway
          > StripeAdapter
          > StripeGateway
      > Builder
      > Factory
      > Singleton

PaymentProcessor.java
1 package DesignPatterns.Adapter;
2
3 public interface PaymentProcessor { no usages 2 implementations
4     void processPayment(double amount); no usages 2 implementations
5 }
```

PayPalGateway.java

```
Project
  cognizant C:\Users\bavag\Downloads\cognizant
    > idea
    > out
    > src
      > DesignPatterns
        > Adapter
          > AdapterPatternTest
          > PaymentProcessor
          > PayPalAdapter
          > PayPalGateway
          > StripeAdapter
          > StripeGateway

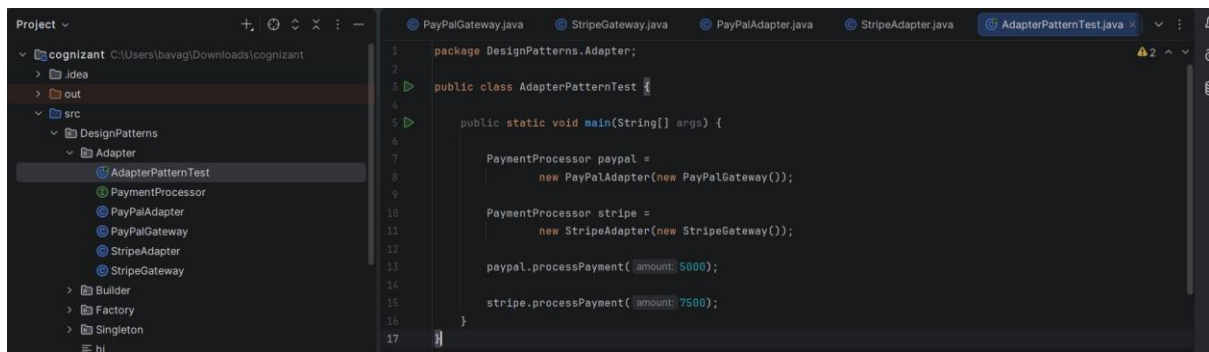
PayPalGateway.java
1 package DesignPatterns.Adapter;
2
3 public class PayPalGateway { no usages
4
5     public void makePayment(double amount) { no usages
6         System.out.println("Payment of Rs." + amount + " processed through PayPal");
7     }
8 }
9
```

StripeGateway.java

```
Project
  cognizant C:\Users\bavag\Downloads\cognizant
    > idea
    > out
    > src
      > DesignPatterns
        > Adapter
          > AdapterPatternTest
          > PaymentProcessor
          > PayPalAdapter
          > PayPalGateway
          > StripeAdapter
          > StripeGateway

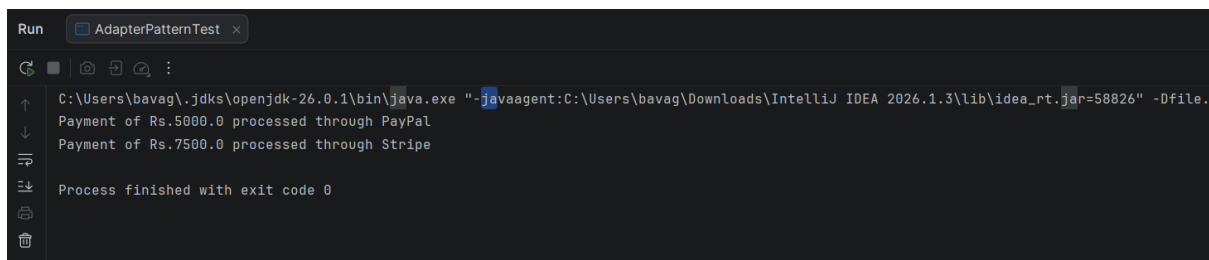
StripeGateway.java
1 package DesignPatterns.Adapter;
2
3 public class StripeGateway { no usages
4
5     public void pay(double amount) { no usages
6         System.out.println("Payment of Rs." + amount + " processed through Stripe");
7     }
8 }
```

PayPalAdapter.java



```
1 package DesignPatterns.Adapter;
2
3 public class AdapterPatternTest {
4
5     public static void main(String[] args) {
6
7         PaymentProcessor paypal =
8             new PayPalAdapter(new PayPalGateway());
9
10        PaymentProcessor stripe =
11            new StripeAdapter(new StripeGateway());
12
13        paypal.processPayment(5000);
14
15        stripe.processPayment(7500);
16    }
17 }
```

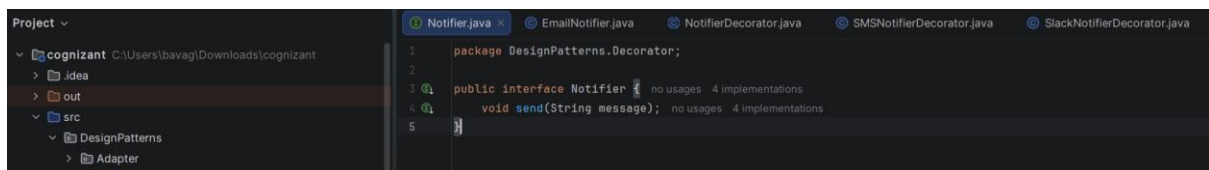
Output



```
Run AdapterPatternTest x
C:\Users\bavag\jdk-26.0.1\bin\java.exe "-javaagent:C:\Users\bavag\Downloads\IntelliJ IDEA 2026.1.3\lib\idea_rt.jar=58826" -Dfile.
Payment of Rs.5000.0 processed through PayPal
Payment of Rs.7500.0 processed through Stripe
Process finished with exit code 0
```

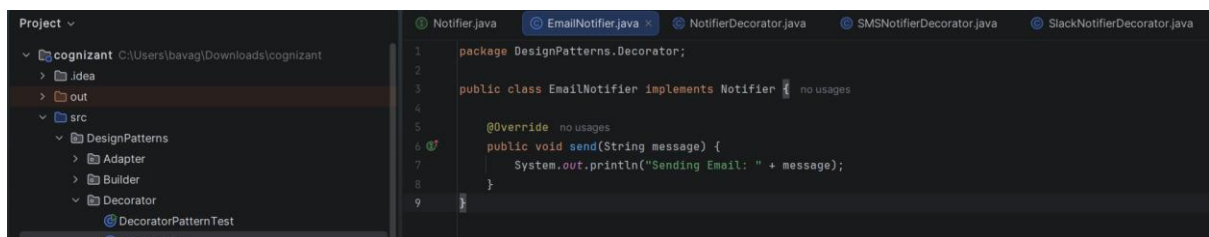
Exercise 5: Implementing the Decorator Pattern

Notifier.java



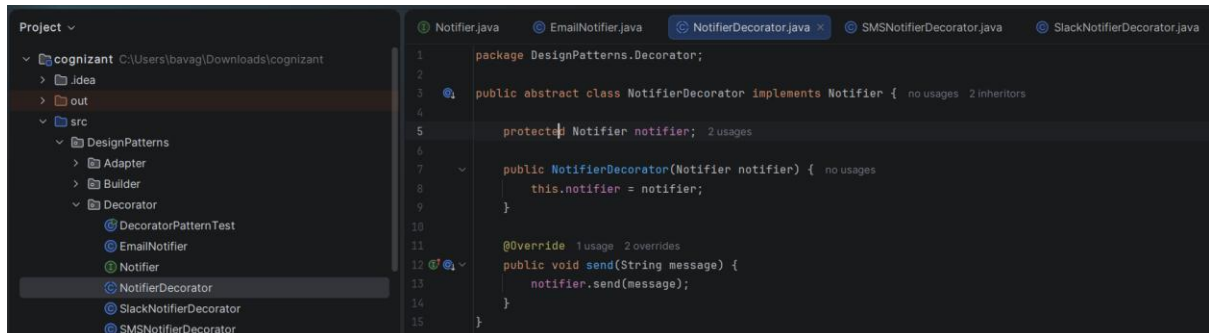
```
1 package DesignPatterns.Decorator;
2
3 public interface Notifier {
4     void send(String message);
5 }
```

EmailNotifier.java



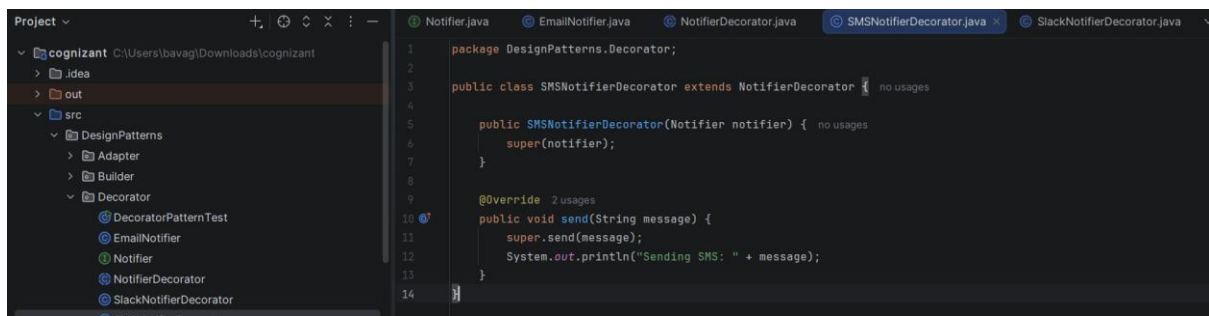
```
1 package DesignPatterns.Decorator;
2
3 public class EmailNotifier implements Notifier {
4
5     @Override
6     public void send(String message) {
7         System.out.println("Sending Email: " + message);
8     }
9 }
```

NotifierDecorator.java



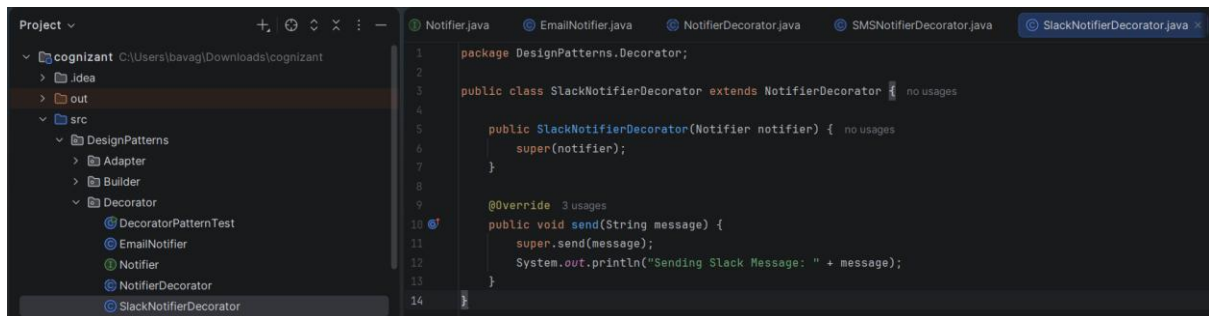
```
1 package DesignPatterns.Decorator;
2
3 public abstract class NotifierDecorator implements Notifier {
4
5     protected Notifier notifier;
6
7     public NotifierDecorator(Notifier notifier) {
8         this.notifier = notifier;
9     }
10
11     @Override
12     public void send(String message) {
13         notifier.send(message);
14     }
15 }
```

SMSNotifierDecorator.java



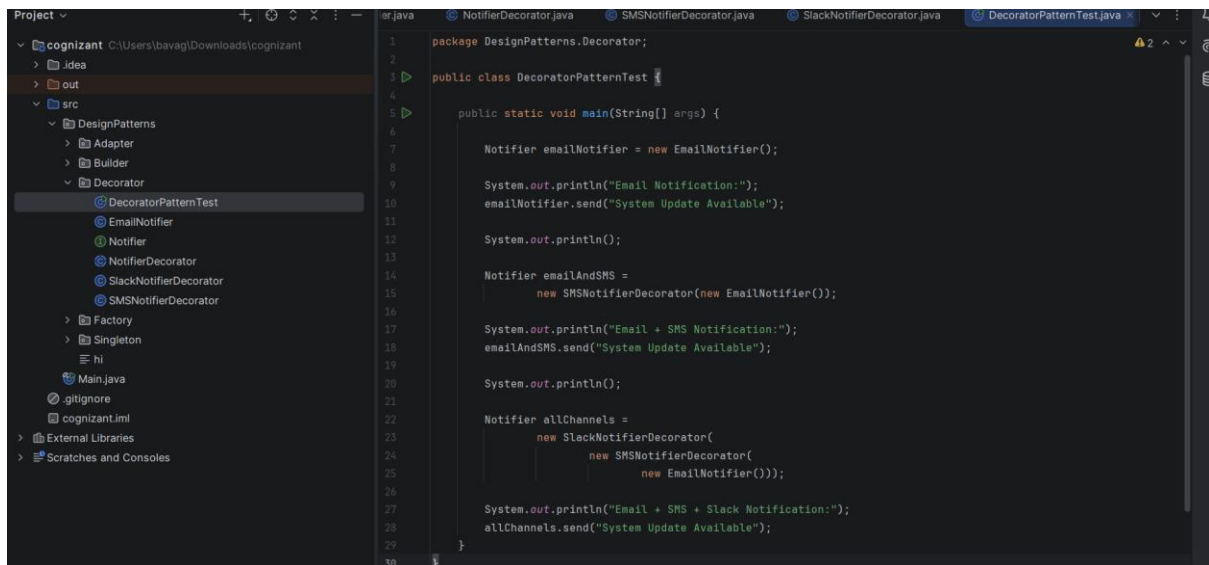
```
1 package DesignPatterns.Decorator;
2
3 public class SMSNotifierDecorator extends NotifierDecorator {
4
5     public SMSNotifierDecorator(Notifier notifier) {
6         super(notifier);
7     }
8
9     @Override
10    public void send(String message) {
11        super.send(message);
12        System.out.println("Sending SMS: " + message);
13    }
14 }
```

SlackNotifierDecorator.java



```
1 package DesignPatterns.Decorator;
2
3 public class SlackNotifierDecorator extends NotifierDecorator {
4
5     public SlackNotifierDecorator(Notifier notifier) {
6         super(notifier);
7     }
8
9     @Override
10    public void send(String message) {
11        super.send(message);
12        System.out.println("Sending Slack Message: " + message);
13    }
14 }
```

DecoratorPatternTest.java



```
1 package DesignPatterns.Decorator;
2
3 public class DecoratorPatternTest {
4
5     public static void main(String[] args) {
6
7         Notifier emailNotifier = new EmailNotifier();
8
9         System.out.println("Email Notification:");
10        emailNotifier.send("System Update Available");
11
12        System.out.println();
13
14        Notifier emailAndSMS =
15            new SMSNotifierDecorator(new EmailNotifier());
16
17        System.out.println("Email + SMS Notification:");
18        emailAndSMS.send("System Update Available");
19
20        System.out.println();
21
22        Notifier allChannels =
23            new SlackNotifierDecorator(
24                new SMSNotifierDecorator(
25                    new EmailNotifier()));
26
27        System.out.println("Email + SMS + Slack Notification:");
28        allChannels.send("System Update Available");
29    }
30 }
```

Output



```
Run DecoratorPatternTest
C:\Users\bavag\jdk-26.0.1\bin\java.exe "-javaagent:C:\Users\bavag\Downloads\IntelliJ IDEA 2026.1.3\lib\idea_rt.jar=49671" -Dfi
Email Notification:
Sending Email: System Update Available

Email + SMS Notification:
Sending Email: System Update Available
Sending SMS: System Update Available

Email + SMS + Slack Notification:
Sending Email: System Update Available
Sending SMS: System Update Available
Sending Slack Message: System Update Available
```

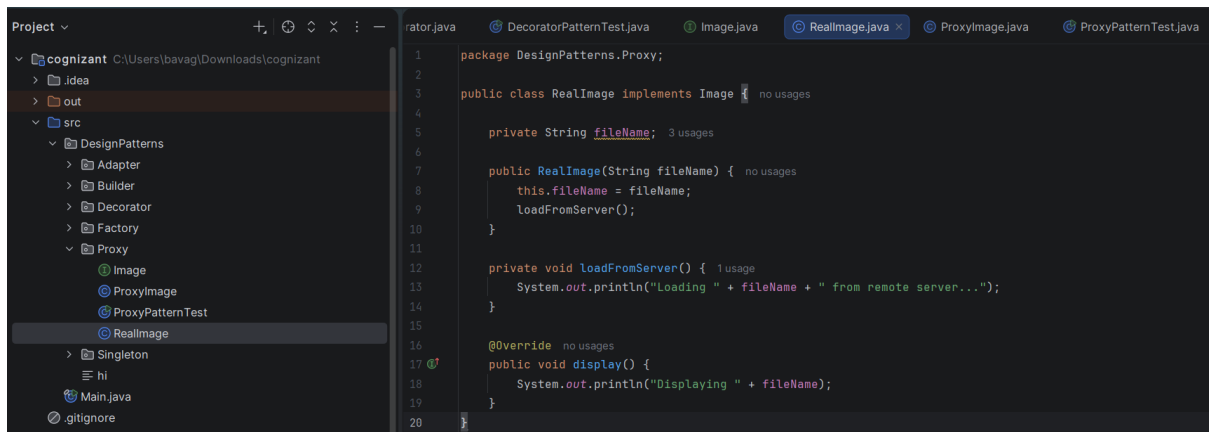

Exercise 6: Implementing the Proxy Pattern

Image.java



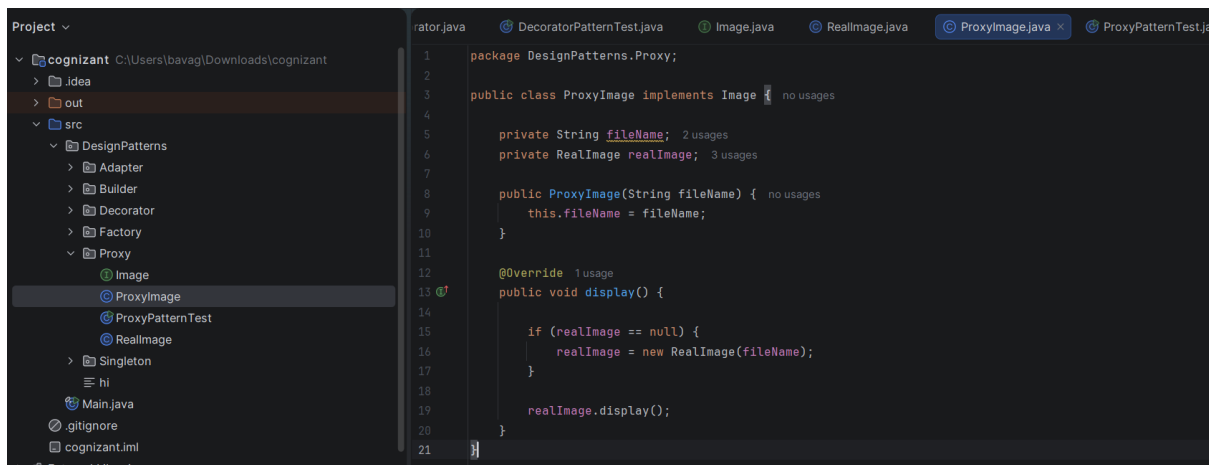
```
1 package DesignPatterns.Proxy;
2
3 public interface Image {
4     void display();
5 }
6
```

RealImage.java



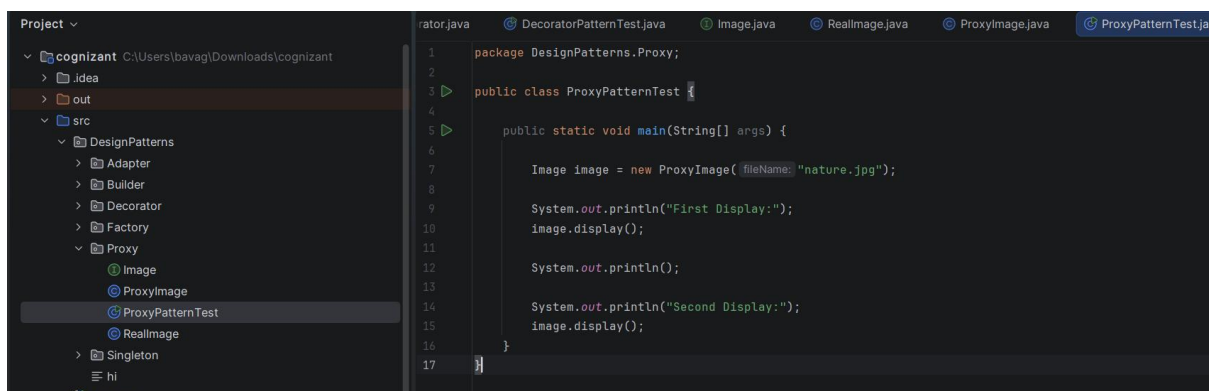
```
1 package DesignPatterns.Proxy;
2
3 public class RealImage implements Image {
4
5     private String fileName;
6
7     public RealImage(String fileName) {
8         this.fileName = fileName;
9         loadFromServer();
10    }
11
12    private void loadFromServer() {
13        System.out.println("Loading " + fileName + " from remote server...");
14    }
15
16    @Override
17    public void display() {
18        System.out.println("Displaying " + fileName);
19    }
20 }
```

ProxyImage.java



```
1 package DesignPatterns.Proxy;
2
3 public class ProxyImage implements Image {
4
5     private String fileName;
6     private RealImage realImage;
7
8     public ProxyImage(String fileName) {
9         this.fileName = fileName;
10    }
11
12    @Override
13    public void display() {
14
15        if (realImage == null) {
16            realImage = new RealImage(fileName);
17        }
18
19        realImage.display();
20    }
21 }
```

ProxyPatternTest.java



```
1 package DesignPatterns.Proxy;
2
3 public class ProxyPatternTest {
4
5     public static void main(String[] args) {
6
7         Image image = new ProxyImage("nature.jpg");
8
9         System.out.println("First Display:");
10        image.display();
11
12        System.out.println();
13
14        System.out.println("Second Display:");
15        image.display();
16    }
17 }
```

Output:

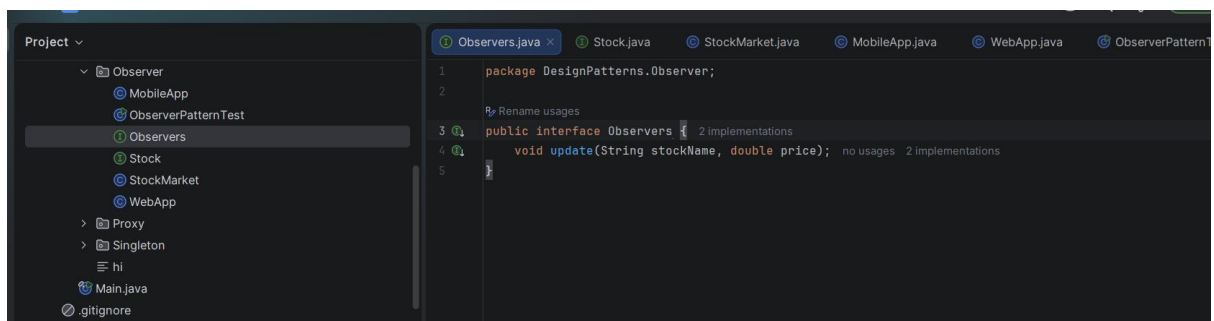
```
C:\Users\bavag\jdk-26.0.1\bin\java.exe "-javaagent:C:\Users\bavag\Downloads\IntelliJ IDEA 2026.1.3\lib\idea_rt.jar=58
First Display:
Loading nature.jpg from remote server...
Displaying nature.jpg

Second Display:
Displaying nature.jpg

Process finished with exit code 0
```

Exercise 7: Implementing the Observer Pattern

Observer.java



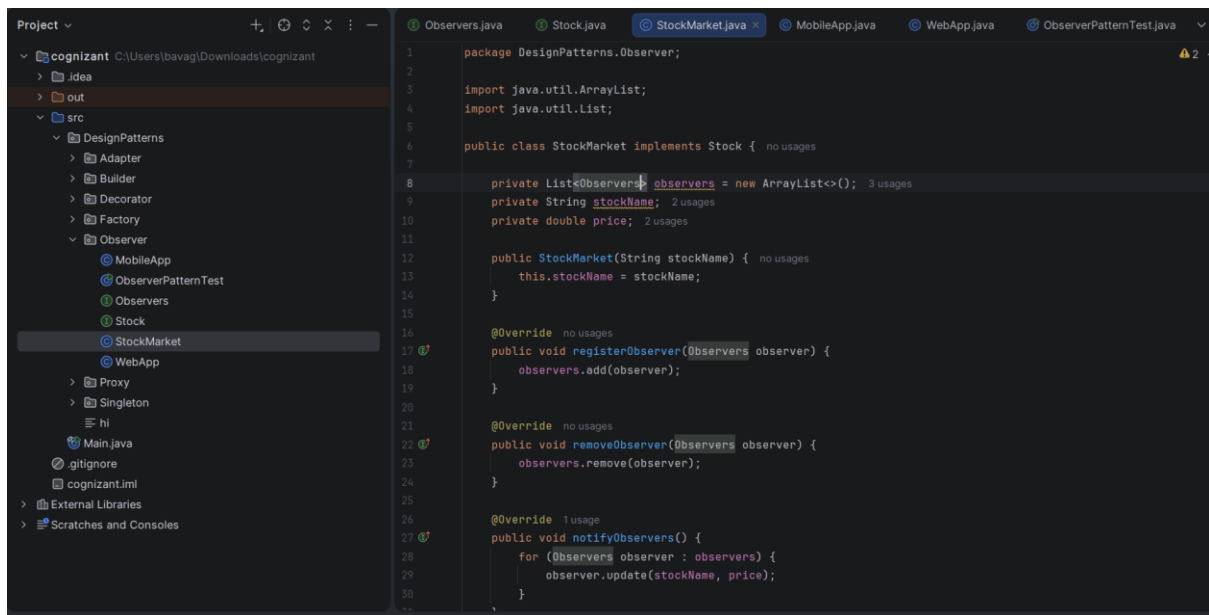
```
1 package DesignPatterns.Observer;
2
3 public interface Observers { 2 implementations
4     void update(String stockName, double price); no usages 2 implementations
5 }
```

Stock.java



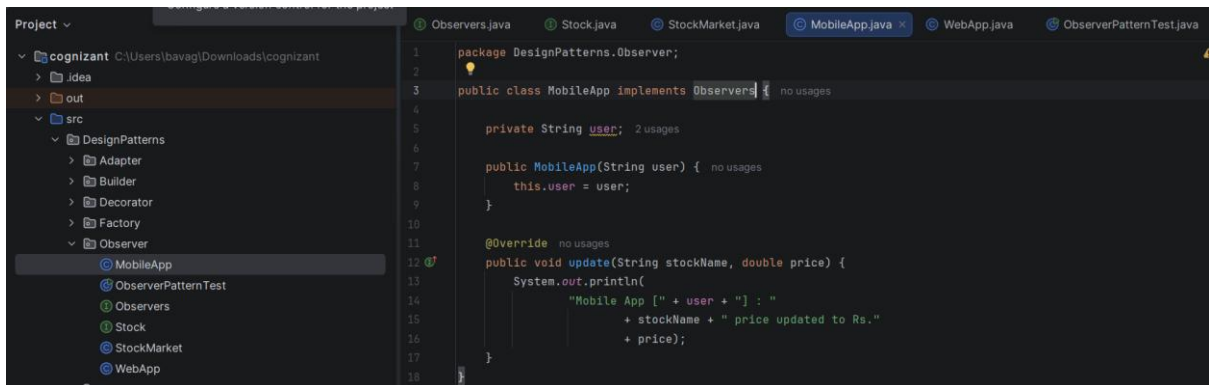
```
1 package DesignPatterns.Observer;
2
3 public interface Stock { no usages 1 implementation
4
5     void registerObserver(Observers observer); no usages 1 implementation
6
7     void removeObserver(Observers observer); no usages 1 implementation
8
9     void notifyObservers(); no usages 1 implementation
10 }
```

StockMarket.java



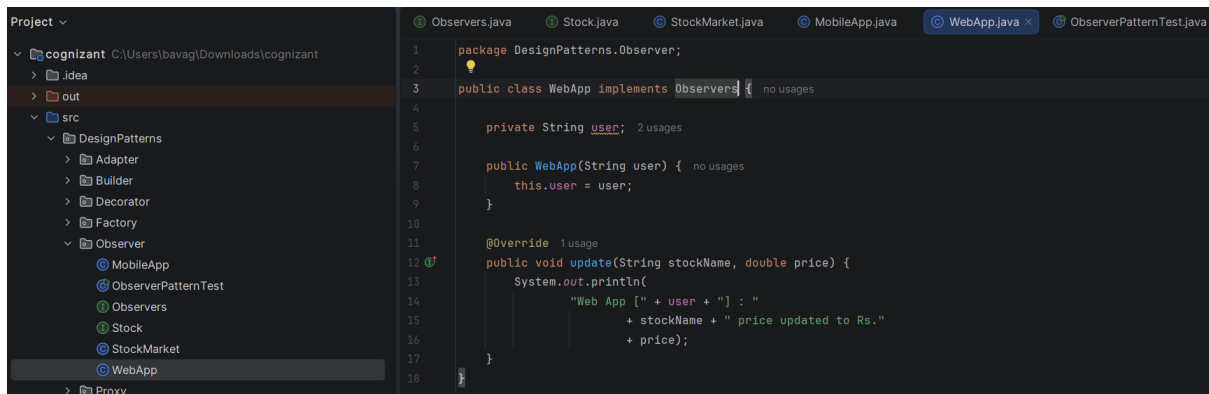
```
1 package DesignPatterns.Observer;
2
3 import java.util.ArrayList;
4 import java.util.List;
5
6 public class StockMarket implements Stock {
7
8     private List<Observers> observers = new ArrayList<>();
9     private String stockName;
10    private double price;
11
12    public StockMarket(String stockName) {
13        this.stockName = stockName;
14    }
15
16    @Override
17    public void registerObserver(Observers observer) {
18        observers.add(observer);
19    }
20
21    @Override
22    public void removeObserver(Observers observer) {
23        observers.remove(observer);
24    }
25
26    @Override
27    public void notifyObservers() {
28        for (Observers observer : observers) {
29            observer.update(stockName, price);
30        }
31    }
32}
```

MobileApp.java



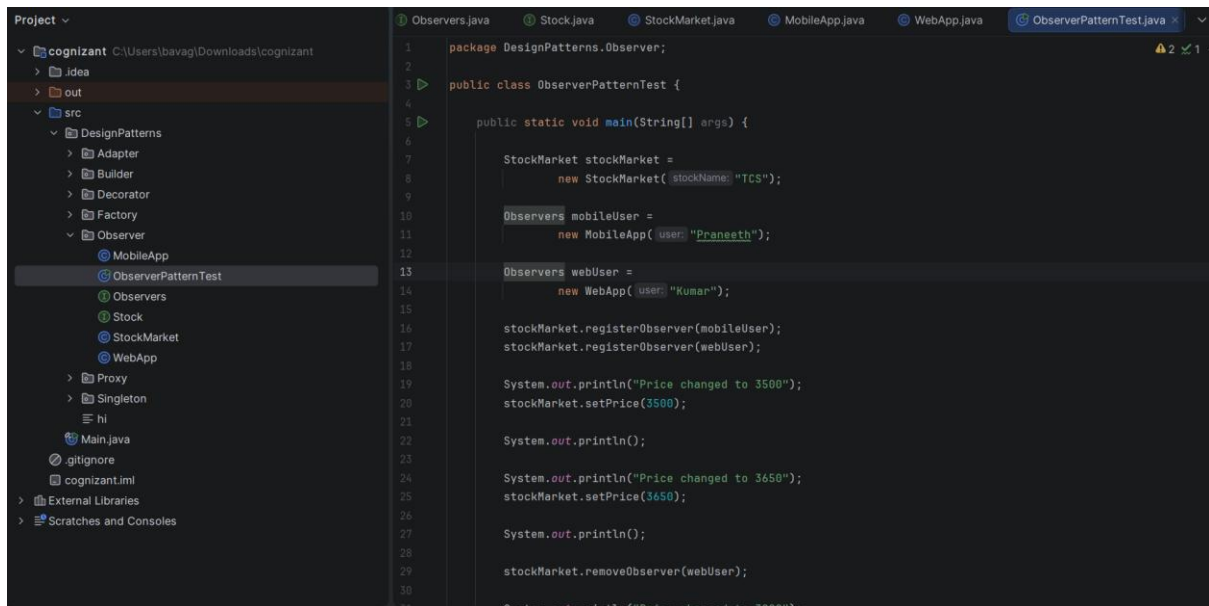
```
1 package DesignPatterns.Observer;
2
3 public class MobileApp implements Observers {
4
5     private String user;
6
7     public MobileApp(String user) {
8         this.user = user;
9     }
10
11    @Override
12    public void update(String stockName, double price) {
13        System.out.println(
14            "Mobile App [" + user + "] : "
15            + stockName + " price updated to Rs."
16            + price);
17    }
18}
```

WebApp.java



```
1 package DesignPatterns.Observer;
2
3 public class WebApp implements Observers {
4
5     private String user;
6
7     public WebApp(String user) {
8         this.user = user;
9     }
10
11    @Override
12    public void update(String stockName, double price) {
13        System.out.println(
14            "Web App [" + user + "] : "
15            + stockName + " price updated to Rs."
16            + price);
17    }
18}
```

ObserverPatternTest.java



```
1 package DesignPatterns.Observer;
2
3 public class ObserverPatternTest {
4
5     public static void main(String[] args) {
6
7         StockMarket stockMarket =
8             new StockMarket("TCS");
9
10        Observers mobileUser =
11            new MobileApp("Praneeth");
12
13        Observers webUser =
14            new WebApp("Kumar");
15
16        stockMarket.registerObserver(mobileUser);
17        stockMarket.registerObserver(webUser);
18
19        System.out.println("Price changed to 3500");
20        stockMarket.setPrice(3500);
21
22        System.out.println();
23
24        System.out.println("Price changed to 3650");
25        stockMarket.setPrice(3650);
26
27        System.out.println();
28
29        stockMarket.removeObserver(webUser);
30    }
31 }
```

output:



```
C:\Users\bavag\jdk-26.0.1\bin\java.exe "-javaagent:C:\Users\bavag\Downloads\IntelliJ IDEA 2026.1.3\lib\idea_rt.jar=51773" -Dfile.encoding=UTF-8
Price changed to 3500
Mobile App [Praneeth] : TCS price updated to Rs.3500.0
Web App [Kumar] : TCS price updated to Rs.3500.0

Price changed to 3650
Mobile App [Praneeth] : TCS price updated to Rs.3650.0
Web App [Kumar] : TCS price updated to Rs.3650.0

Price changed to 3800
Mobile App [Praneeth] : TCS price updated to Rs.3800.0
```

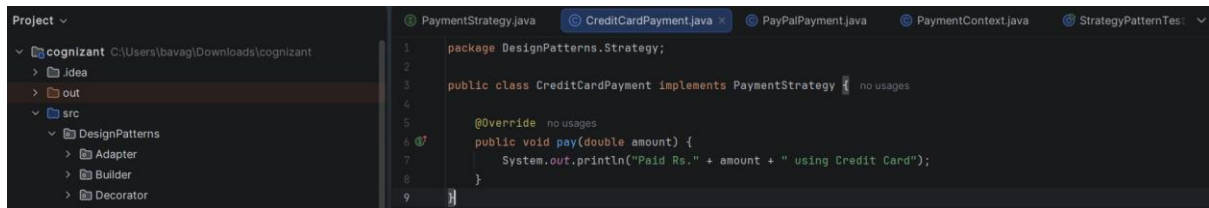
Exercise 8: Implementing the Strategy Pattern

PaymentStrategy.java



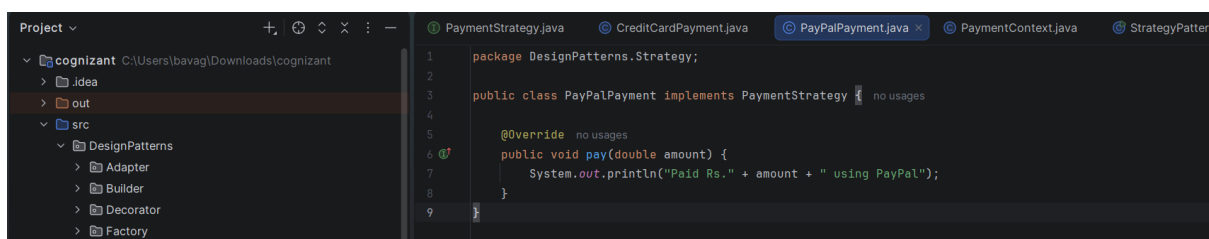
```
1 package DesignPatterns.Strategy;
2
3 public interface PaymentStrategy {
4     void pay(double amount);
5 }
```

CreditCardPayment.java



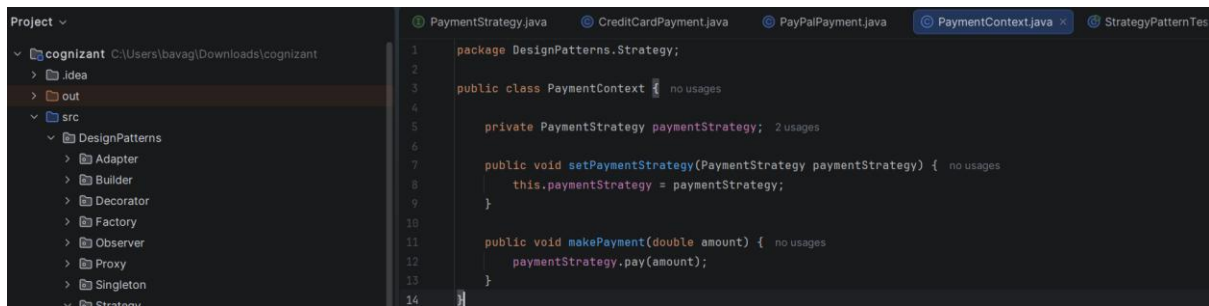
```
1 package DesignPatterns.Strategy;
2
3 public class CreditCardPayment implements PaymentStrategy {
4
5     @Override
6     public void pay(double amount) {
7         System.out.println("Paid Rs." + amount + " using Credit Card");
8     }
9 }
```

PayPalPayment.java



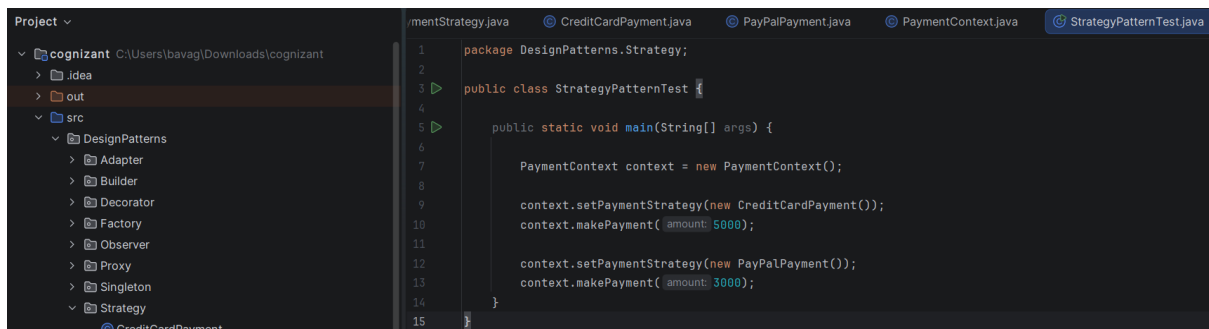
```
1 package DesignPatterns.Strategy;
2
3 public class PayPalPayment implements PaymentStrategy {
4
5     @Override
6     public void pay(double amount) {
7         System.out.println("Paid Rs." + amount + " using PayPal");
8     }
9 }
```

PaymentContext.java



```
1 package DesignPatterns.Strategy;
2
3 public class PaymentContext {
4
5     private PaymentStrategy paymentStrategy;
6
7     public void setPaymentStrategy(PaymentStrategy paymentStrategy) {
8         this.paymentStrategy = paymentStrategy;
9     }
10
11     public void makePayment(double amount) {
12         paymentStrategy.pay(amount);
13     }
14 }
```

StrategyPatternTest.java



```
1 package DesignPatterns.Strategy;
2
3 public class StrategyPatternTest {
4
5     public static void main(String[] args) {
6
7         PaymentContext context = new PaymentContext();
8
9         context.setPaymentStrategy(new CreditCardPayment());
10        context.makePayment(5000);
11
12        context.setPaymentStrategy(new PayPalPayment());
13        context.makePayment(3000);
14    }
15 }
```

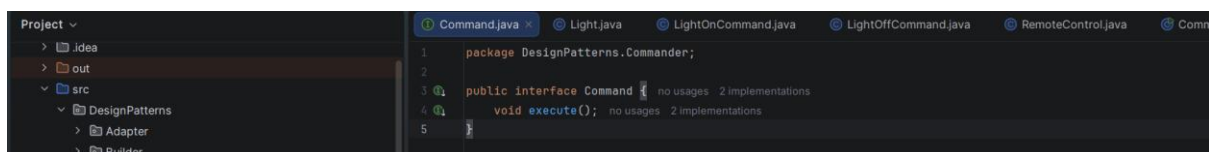
Output:



```
C:\Users\bavag\.jdk\openjdk-26.0.1\bin\java.exe "-javaagent:C:\Users\bavag\Downloads\IntelliJ IDEA 2026.1.3\lib\idea_rt.jar=64099" -Dfile.encoding=UTF-8
Paid Rs.5000.0 using Credit Card
Paid Rs.3000.0 using PayPal
Process finished with exit code 0
```

Exercise 9: Implementing the Command Pattern

Command.java



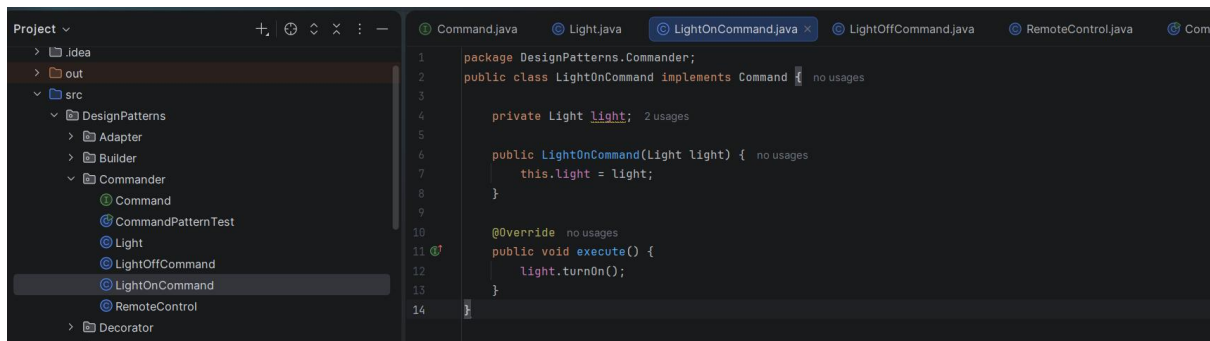
```
1 package DesignPatterns.Commander;
2
3 public interface Command {
4     void execute();
5 }
```

Light.java



```
1 package DesignPatterns.Commander;
2
3 public class Light {
4
5     public void turnOn() {
6         System.out.println("Light is ON");
7     }
8
9     public void turnOff() {
10        System.out.println("Light is OFF");
11    }
12 }
```

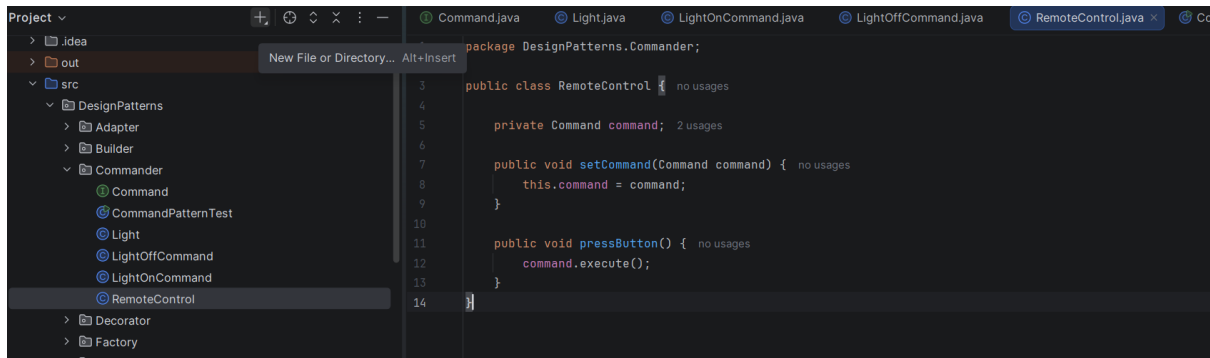
LightOnCommand.java



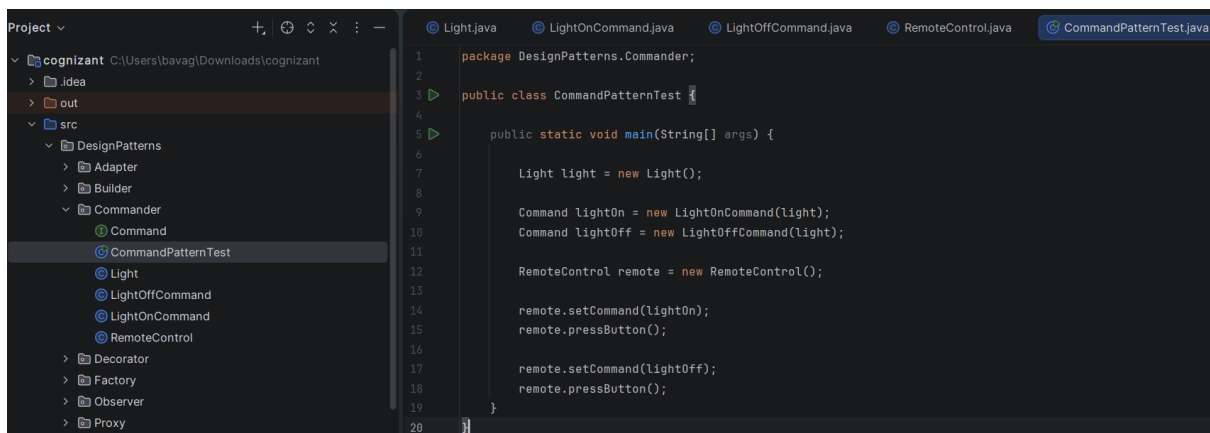
LightOffCommand.java



RemoteControl.java



CommandPatternTest.java



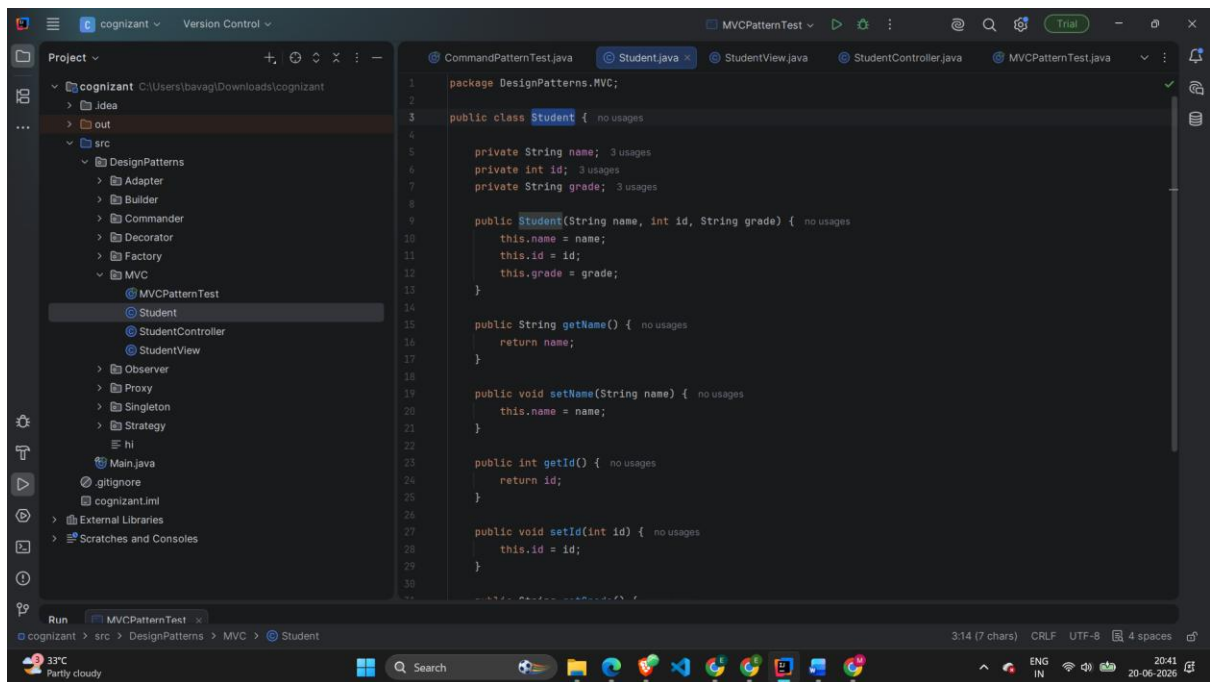
Output:

```
C:\Users\bavag\.jdk\openjdk-26.0.1\bin\java.exe "-javaagent:C:\Users\bavag\Downloads\IntelliJ IDEA 2026.1.3\lib\idea_rt.jar=64099
Paid Rs.5000.0 using Credit Card
Paid Rs.3000.0 using PayPal

Process finished with exit code 0
```

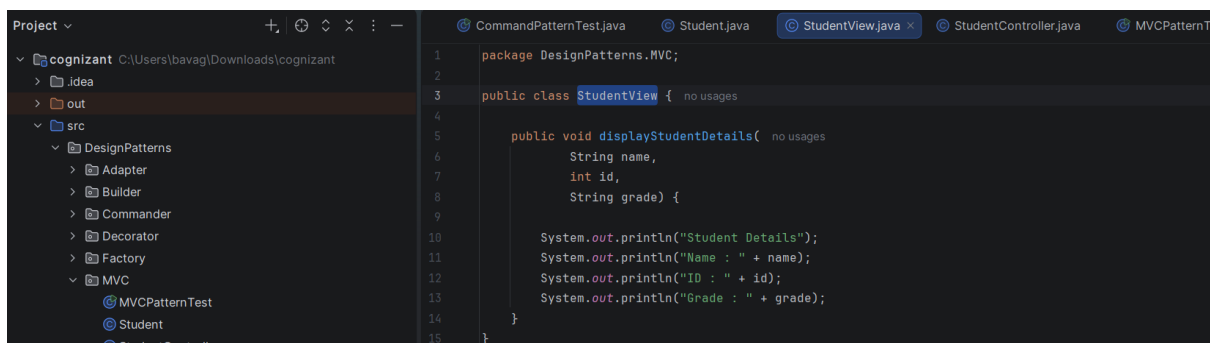
Exercise 10: Implementing the MVC Pattern

Student



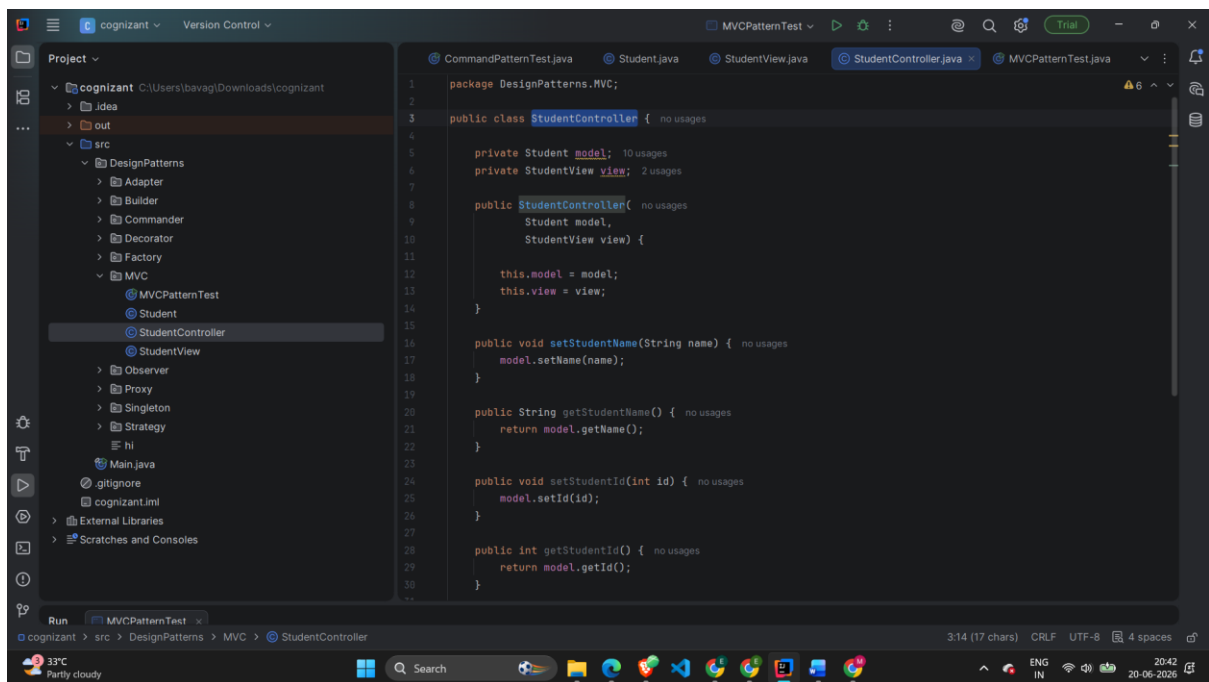
```
1 package DesignPatterns.MVC;
2
3 public class Student { no usages
4
5     private String name; 3 usages
6     private int id; 3 usages
7     private String grade; 3 usages
8
9     public Student(String name, int id, String grade) { no usages
10         this.name = name;
11         this.id = id;
12         this.grade = grade;
13     }
14
15     public String getName() { no usages
16         return name;
17     }
18
19     public void setName(String name) { no usages
20         this.name = name;
21     }
22
23     public int getId() { no usages
24         return id;
25     }
26
27     public void setId(int id) { no usages
28         this.id = id;
29     }
30 }
```

StudentView.java

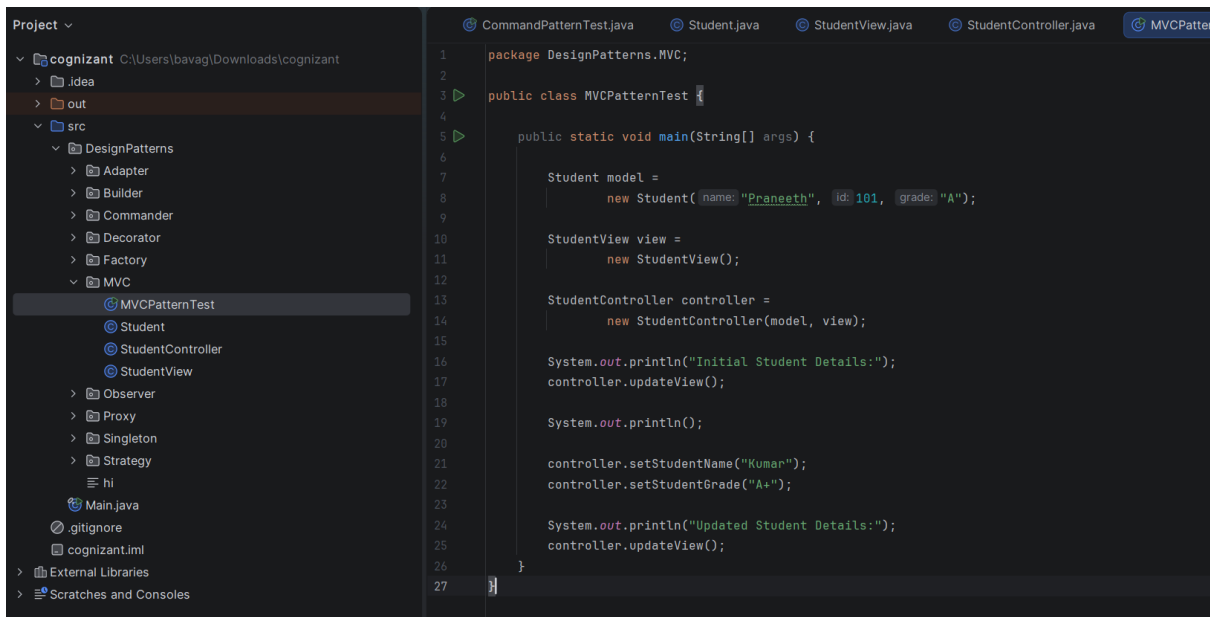


```
1 package DesignPatterns.MVC;
2
3 public class StudentView { no usages
4
5     public void displayStudentDetails( no usages
6         String name,
7         int id,
8         String grade) {
9
10         System.out.println("Student Details");
11         System.out.println("Name : " + name);
12         System.out.println("ID : " + id);
13         System.out.println("Grade : " + grade);
14     }
15 }
```

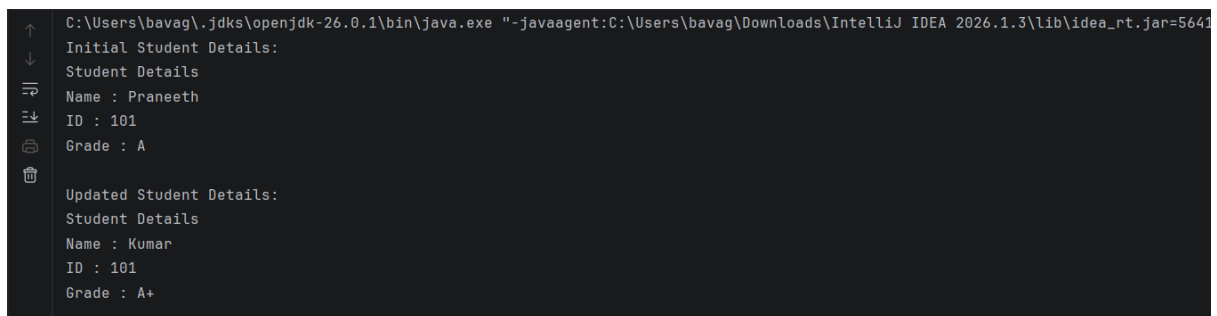
StudentController.java



Test



Output:

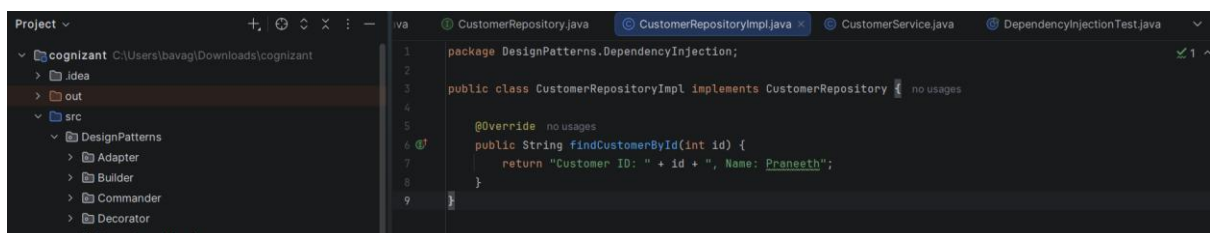


Exercise 11: Implementing Dependency Injection

CustomerRepository.java



CustomerRepositoryImpl.java

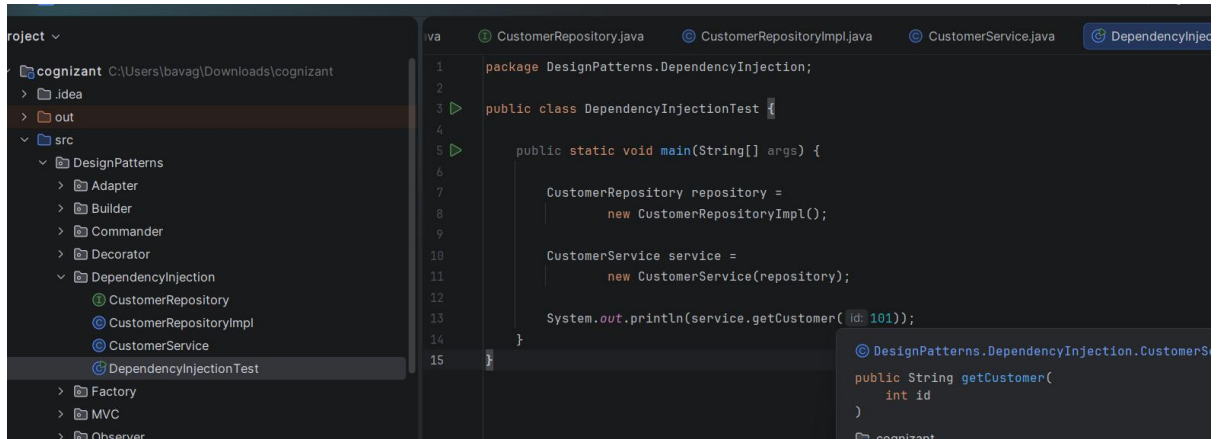


CustomerService.java



```
1 package DesignPatterns.DependencyInjection;
2
3 public class CustomerService {
4
5     private CustomerRepository customerRepository;
6
7     public CustomerService(CustomerRepository customerRepository) {
8         this.customerRepository = customerRepository;
9     }
10
11     public String getCustomer(int id) {
12         return customerRepository.findCustomerById(id);
13     }
14 }
```

DependencyInjectionTest.java



```
1 package DesignPatterns.DependencyInjection;
2
3 public class DependencyInjectionTest {
4
5     public static void main(String[] args) {
6
7         CustomerRepository repository =
8             new CustomerRepositoryImpl();
9
10        CustomerService service =
11            new CustomerService(repository);
12
13        System.out.println(service.getCustomer(101));
14    }
15 }
```

Output:



```
C:\Users\bavag\jdk\openjdk-26.0.1\bin\java.exe "-javaagent:C:\Users\bavag\Downloads\IntelliJ IDEA 2026.1.3\lib\idea_rt.jar-64794" -Dfile.encoding=UTF-8
Customer ID: 101, Name: Praneeth
Process finished with exit code 0
```